

Object-Oriented Programming Using

C++



Lesson N7

Object-Oriented Programming Using C++

Contents

Concept of the Dynamic Data Structure	3
Stack.....	5
Basic Operations Performed on Stack.....	5
Implementation — Stack.....	6
Queue.....	10
Implementation — Queue	11
Circular Queue.....	15
Implementation — Circular Queue.....	15
Priority Queue	19
Implementation — Priority Queue	20
Homework.....	26

Concept of the Dynamic Data Structure

Today we are introduced to the concept of the dynamic data structure.

Up to this point we have been working only with data that have a static structure, unchanging during the execution of the program.

Paradoxically, but even a dynamic array essentially is not very dynamic, because in order to change its size, you should re-create it.

In other words, when the program is working, it's quite simple to change only the values of the elements, while a change of the amount of these elements leads to a number of monotonic procedures.

And this is, of course, not always convenient.

Assume that in the program for entering and processing data about the pupils of class, static arrays are used to store data.

In determining the size of the array the programmer has to rely on the average or limit number of students in the class.

In this case, if the real number of pupils in the class is less than the assumed number, a computer's memory is used inefficiently, but if this number is greater, the program can't be used (as it is necessary to make changes to the source code and compile). Creating a dynamic array will be a solution undoubtedly, but will lead to a complication of writing code.

Tasks processing data, which by their nature are dynamic, it is convenient to solve with the help of dynamic structures. We will learn how to work with this interesting tool.

Considering all the above, we can assumed that the dynamic structures are some structures, capable, if necessary, to allocate memory for new elements or delete the allocated memory for unnecessary elements when the program works.

For solving the problem of addressing dynamic data structures we use a method called *dynamic memory allocation*, that is, a memory for the separate elements is allocated at a time when they "begin to exist" in the process of the implementation of the program, but not at compile time.

The compiler in this case only allocates a fixed amount of memory for storing the address of dynamically placed element but not of the element itself.

There are several types of dynamic data structures. In addition, each of them has both advantages and disadvantages. So, what type to use, depends entirely on the specific task.

Stack

Stack is a dynamic data structure this is an ordered set of elements. In this structure new items are added and existing items are removed from one end that is called a top of the stack.

Moreover, the stack has a so called base address. Under this concept lies Restart Kicks memory address in which the stack is placed.

By definition, elements are inserted and removed according to the *last-in first-out* (LIFO) principle.

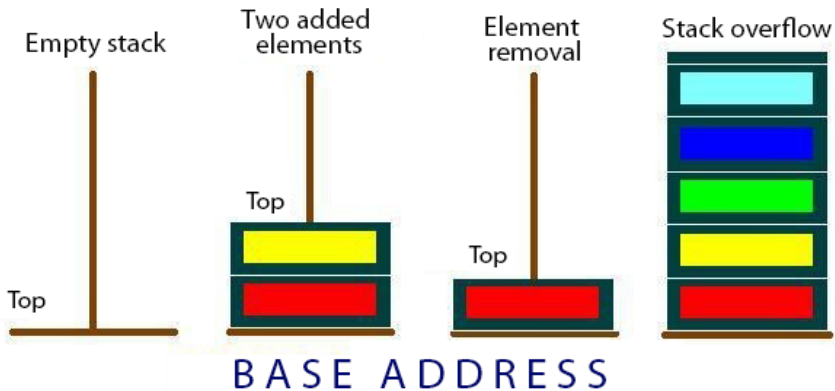
The most obvious example of the organization of the stack is a children's pyramid, where the addition and removal of rings is carried out exactly according to the definition of the stack.

Take a note!!! For stack storage in the memory a solid area is allocated, the boundary addresses of this area are the physical parameters of the stack structure. If in the process of filling of the stack a pointer moves "up" and goes beyond the boundaries of the originally allotted area, a stack overflow occurs. A stack overflow is regarded as an exceptional situation that requires taking measures for its elimination.

Basic Operations Performed on Stack

1. The procedure of inserting a new element to the top of the stack is known as **Push Operation**
2. The procedure of removing element from the top of the stack is called Pop Operation.
3. Check whether the stack is empty. (The stack is considered "empty" if the top pointer coincides with the pointer of the lower boundary).

4. Viewing item on the top of the stack without removing.
5. Stack cleaning.



Applications: Dynamic structure Stack is most commonly used when parsing all kinds of expressions.

Note. By the way, each application has its own stack. In it the compiler creates local variables. And a transfer of parameters to the function is implemented through the stack.

Implementation — Stack

```
#include <iostream>
#include <string.h>
#include <time.h>
using namespace std;
class Stack
{
    //Lower boundary and upper boundary of the stack
    enum {EMPTY = -1, FULL = 20};

    //Array for storing data
    char st[FULL + 1];
```

```

        //Pointer to the top of the stack
        int top;

public:
    //Constructor
    Stack();

    //Add item
    void Push(char c);

    //Extract the element
    char Pop();

    //Stack cleaning
    void Clear();

    //Check the existence of items in the stack
    bool IsEmpty();

    //Check the stack overflow
    bool IsFull();

    //Number of items in the stack
    int GetCount();
};

Stack::Stack()
{
    //Initially the stack is empty
    top = EMPTY;
}

void Stack::Clear()
{
    //Effective "cleaning" of the stack (data
    //are still in the array, but the functions
    //of the class, that are oriented to the work
    //with the top of the stack, will ignore them)
    top = EMPTY;
}

```

```
bool Stack::IsEmpty()
{
    //Is it empty?
    return top == EMPTY;
}

bool Stack::IsFull()
{
    //Is it full?
    return top == FULL;
}

int Stack::GetCount()
{
    //Number of items present in the stack
    return top + 1;
}

void Stack::Push(char c)
{
    //If there is a place in the stack, we increase the pointer
    //to the top of the stack and insert a new element
    if(!IsFull())
        st[++top] = c;
}

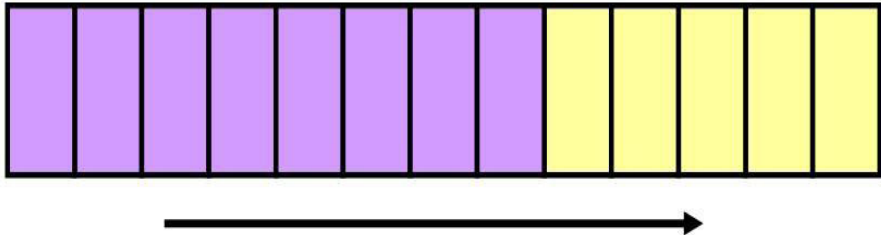
char Stack::Pop()
{
    //If there are elements in the stack, the upper
    //we return and reduce the pointer
    //to the top of the stack
    if(!IsEmpty())
        return st[top--];
    else //If there are not items in the stack
        return 0;
}
```



```
void main()
{
    srand(time(0));
    Stack ST;
    char c;
    //until the stack is filled
    while(!ST.IsFull()){
        c=rand()%4+2;
        ST.Push(c);
    }
    //until the stack is released
    while(c=ST.Pop()){
        cout<<c<<" ";
    }
    cout<<"\n\n";
}
```

Queue

Simple queue is a next dynamic structure, which we will consider. Queue just like a stack may be implemented on practice by using an array. Queue is a coherent set of elements of variable length. In this case new additions to a line made to the back of the queue, while removal (or serving) happens in the front. Items are inserted and removed according to the first-in first-out (FIFO) principle. For a queue we allocate a finite sequence of elements, from which in each current moment of the time elements of the queue fill only a part of sequential elements.



Basically, you constantly face with a simple queue.

Here are just some of the examples: queue in mausoleum, ***the printer print queue, even the action of the linear algorithm executed in turn.***

Implementation — Queue

```
#include <iostream>
#include <string.h>
#include <time.h>
using namespace std;

class Queue
{
    //Queue
    int * Wait;
    //Maximum length of the queue
    int MaxQueueLength;
    //Current length of the queue
    int QueueLength;

public:
    //Constructor
    Queue(int m);

    //Destructor
    ~Queue();

    //Insert an element into queue
    void Add(int c);

    //Extract the element
    int Extract();

    //Queue cleaning
    void Clear();

    //Check the existence of items in the queue
    bool IsEmpty();

    //Check the queue overflow
    bool IsFull();
```

```

        //Number of elements in the queue
        int GetCount();

        //Display the queue elements
        void Show();
};

void Queue::Show() {
    cout<<"\n-----\n";
    //Queue demonstration
    for(int i=0;i<QueueLength;i++){
        cout<<Wait[i]<<" ";
    }
    cout<<"\n-----\n";
}

Queue::~~Queue()
{
    //Delete the queue
    delete[]Wait;
}

Queue::Queue(int m)
{
    //Get the length
    MaxQueueLength=m;
    //Create the queue
    Wait=new int[MaxQueueLength];
    //Initially the queue is empty
    QueueLength = 0;
}

void Queue::Clear()
{
    //Effective "cleaning" of the queue
    QueueLength = 0;
}

```

```

bool Queue::IsEmpty()
{
    //Is it empty?
    return QueueLength == 0;
}

bool Queue::IsFull()
{
    //Is it full?
    return QueueLength == MaxQueueLength;
}

int Queue::GetCount()
{
    //Number of elements present in the stack
    return QueueLength;
}

void Queue::Add(int c)
{
    //If there is a free space in the queue, we increase
    //the number of values and insert a new element
    if(!IsFull())
        Wait[QueueLength++] = c;
}

int Queue::Extract()
{
    //If there are items in the queue, we return the one
    //that entered the first and we shift the queue
    if(!IsEmpty()){
        //Remember the first
        int temp=Wait[0];

        //Shift all items
        for(int i=1;i<QueueLength;i++)
            Wait[i-1]=Wait[i];
    }
}

```

```
        //Reduce the number
        QueueLength--;

        //Return the first (null)
        return temp;
    }

    else //If there aren't elements in the stack
        return -1;
}

void main()
{
    srand(time(0));

    //Create the queue
    Queue QU(25);

    //Fill a part of elements
    for(int i=0;i<5;i++){
        QU.Add(rand()%50);
    }
    //Display the queue elements
    QU.Show();

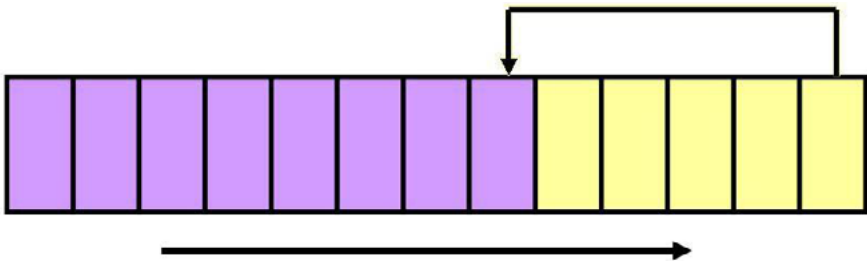
    //Extract the element
    QU.Extract();

    //Display the queue elements
    QU.Show();
}
```

Circular Queue

Circular queue is a very similar to the simple queue. It is also built on the ideology of the FIFO, we recall — *the first element that has been added to the queue will be the first which will leave a queue.*

The only difference is that the element that comes out of the beginning of the queue will be moved to the end of the queue.



We can give a very simple example of known to you from childhood the water cycle, or the trams plying on a circular route.

Implementation — Circular Queue

```
#include <iostream>
#include <string.h>
#include <time.h>
using namespace std;

class QueueRing
{
    //Queue
    int * Wait;
    //Maximum length of the queue
    int MaxQueueLength;
```

```

        //Current length of the queue
        int QueueLength;

public:
    //Constructor
    QueueRing(int m);

    //Destructor
    ~QueueRing();

    //Insert an element into queue
    void Add(int c);

    //Extract the element
    bool Extract();

    //Queue cleaning
    void Clear();

    //Check the existence of items in the queue
    bool IsEmpty();

    //Check the queue overflow
    bool IsFull();

    //Number of elements in the queue
    int GetCount();

    //Display the queue elements
    void Show();
};

void QueueRing::Show() {
    cout<<"\n-----\n";
    //Display the queue elements
    for(int i=0;i<QueueLength;i++){
        cout<<Wait[i]<<" ";
    }
}

```



```

cout<<"\n-----\n";
}

QueueRing::~QueueRing()
{
    //Delete the queue
    delete[]Wait;
}

QueueRing::QueueRing(int m)
{
    //Get the length
    MaxQueueLength=m;
    //Create the queue
    Wait=new int[MaxQueueLength];
    //Initially the queue is empty
    QueueLength = 0;
}

void QueueRing::Clear()
{
    //Effective "cleaning" of the queue
    QueueLength = 0;
}

bool QueueRing::IsEmpty()
{
    //Is it empty?
    return QueueLength == 0;
}

bool QueueRing::IsFull()
{
    //Is it full?
    return QueueLength == MaxQueueLength;
}

```

```

int QueueRing::GetCount()
{
    //Number of elements present in the stack
    return QueueLength;
}

void QueueRing::Add(int c)
{
    //If there is a free space in the queue,
    //we increase the number of values
    //and insert a new element
    if(!IsFull())
        Wait[QueueLength++] = c;
}

bool QueueRing::Extract()
{
    //If there are items in the queue, we return the one
    //that entered the first and we shift the queue
    if(!IsEmpty()){
        //Remember the first
        int temp=Wait[0];

        //Shift all items
        for(int i=1;i<QueueLength;i++)
            Wait[i-1]=Wait[i];

        //We throw the first "ejected element
        //to the end " of the queue.
        Wait[QueueLength-1]=temp;
        return 1;
    }
    else return 0;
}

void main()
{
    srand(time(0));

```

```

//Create the queue
QueueRing QUR(25);

//Fill a part of elements
for(int i=0;i<5;i++){
    QUR.Add(rand()%50);
}
//Display the queue elements
QUR.Show();

//Extract the element
QUR.Extract();

//Display the queue elements
QUR.Show();
}

```

Priority Queue

We have already got acquainted with two types of queues and they were simple enough. However, there is one more queue — **priority queue**.

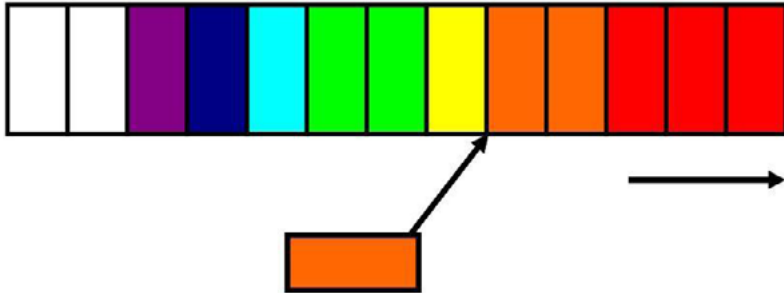
The fact is that it is often necessary to create a queue, where (excuse for the tautology) the order of the output depends on the priority elements.

Any number or temporary constant can be a priority. At the time of the extraction we will select that element, which has a higher priority.

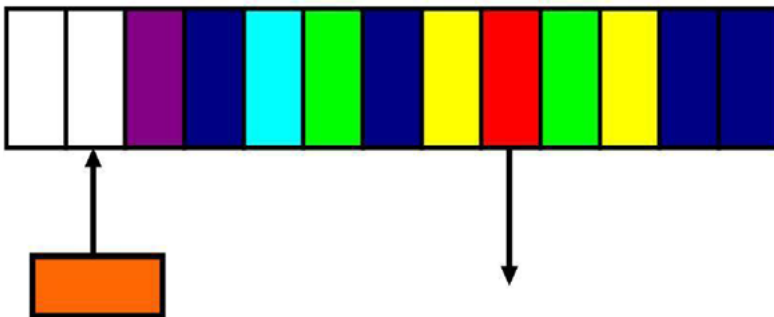
There are several kinds of priority queues:

1. **The queue with priority inclusion.** A sequence of the queue elements is strictly ordered. In other words, each element at the time of entering the queue is placed immediately

in accordance with its priority. When some element is excluded, an item from the beginning is extracted.



2. **Queues with priority exclusion.** An element is added to the end of the queue, and we extract the most priority element, which is subsequently removed from the queue.



Implementation — Priority Queue

```
#include <iostream>
#include <string.h>
#include <time.h>
using namespace std;
class QueuePriority
{
```

```

    //Queue
    int * Wait;
    //Priority
    int * Pri;
    //Maximum length of the queue
    int MaxQueueLength;
    //Current length of the queue
    int QueueLength;

public:
    //Constructor
    QueuePriority(int m);

    //Destructor
    ~QueuePriority();

    //Insert an element into queue
    void Add(int c,int p);

    //Extract the element
    int Extract();

    //Queue cleaning
    void Clear();

    //Check the existence of items in the queue
    bool IsEmpty();

    //Check the queue overflow
    bool IsFull();

    //Number of elements in the queue
    int GetCount();

    //Display the queue elements
    void Show();
};

```

```

void QueuePriority::Show() {
    cout<<"\n-----\n";
    //Display the queue elements
    for(int i=0;i<QueueLength;i++)
        { cout<<Wait[i]<<" - "<<Pri[i]<<"\n\n";
        }
    cout<<"\n-----\n";
}

QueuePriority::~QueuePriority()
{
    //Delete the queue
    delete[]Wait;
    delete[]Pri;
}

QueuePriority::QueuePriority(int m)
{
    //get the length
    MaxQueueLength=m;
    //create the queue
    Wait=new int[MaxQueueLength];
    Pri=new int[MaxQueueLength];
    //Initially the queue is empty
    QueueLength = 0;
}

void QueuePriority::Clear()
{
    //Effective "cleaning" of the queue
    QueueLength = 0;
}

bool QueuePriority::IsEmpty()
{
    //Is it empty?
    return QueueLength == 0;
}

```

```

bool QueuePriority::IsFull()
{
    //Is it full?
    return QueueLength == MaxQueueLength;
}

int QueuePriority::GetCount()
{
    //Number of elements present in the stack
    return QueueLength;
}

void QueuePriority::Add(int c,int p)
{
    //If there is a free space in the queue, we increase
    //the number of values and insert a new element
    if(!IsFull()){
        Wait[QueueLength] = c;
        Pri[QueueLength] = p;
        QueueLength++;
    }
}

int QueuePriority::Extract()
{
    //If there are items in the queue, we return
    //the one that has the highest priority
    //and we shift the queue
    if(!IsEmpty()){
        //Let zero element is a priority
        int max_pri=Pri[0];
        //and priority index = 0
        int pos_max_pri=0;

        //We are looking for a priority
        for(int i=1;i<QueueLength;i++)

```

```

        //We found a higher priority element
        if(max_pri<Pri[i]){
            max_pri=Pri[i];
            pos_max_pri=i;
        }
        //Extract a priority item
        int temp1=Wait[pos_max_pri];
        int temp2=Pri[pos_max_pri];

        //Shift all elements
        for(int i=pos_max_pri;i<QueueLength-1;i++){
            Wait[i]=Wait[i+1];
            Pri[i]=Pri[i+1];
        }
        //Reduce the number
        QueueLength--;
        //Return the extracted element
        return temp1;

    }
    else return -1;
}

void main()
{
    srand(time(0));

    //Create the queue
    QueuePriority QUP(25);
    //Fill a part of elements
    for(int i=0;i<5;i++){

        //Values from 0 to 99 (inclusive)
        //and priority from 0 to 11 (inclusive)
        QUP.Add(rand()%100,rand()%12);
    }
}

```



```
//Display the queue elements  
QUP.Show();  
  
//Extract the element  
QUP.Extract();  
  
//Display the queue elements  
QUP.Show();  
}
```

Homework

1. Create an imitation of the game "one-armed bandit". For example, when you press the "Enter" happening "rotation" of three drums (of course, the number of rotations of each of them is randomly chosen), on which different icons are depicted; and if a certain combination is got, the player gets some win.
2. Create an imitation model "minibuses stop". You must enter the following information : the average time between occurrences of passengers at the bus stop at different times of the day , the average time between the occurrences of minibuses at the bus stop at different times of the day, a type of the stop (the last or not). It is necessary to determine: the average time of stay of the person at the bus stop, sufficient interval of the time between arrivals of minibuses, at stop not more than N people simultaneously should be. Number of free seats in the minibus is a random variable.
3. Develop an application that simulates the queue print of the printer. Enter clients that send requests to the printer, each of which has its own priority. Every new client enters the queue according to its priority. It is necessary to preserve the print statistics (user, time) in a separate queue. Provided the output statistics to the screen.

